

Đại học Quốc gia Thành phố Hồ Chí Minh
Trường Đại học Bách khoa
Khoa Khoa học và Kỹ thuật Máy tính

CHUYÊN ĐỀ 1 TIỀN SĨ

Nghiên cứu về các Giải pháp Học máy trong việc Cải thiện hiệu quả Quá trình Định thời trên các Hệ thống Tính toán Hiệu năng cao

Họ tên NCS: Hoàng Lê Hải Thanh
Khóa: 242
Mã số NCS: 2480861
Ngành: Khoa học Máy tính
Khoa: Khoa học và Kỹ thuật Máy tính
CBHD 1: PGS.TS. Thoại Nam
CBHD 2: TS. Nguyễn Lê Duy Lai

Hệ thống HPC đang phục vụ workload ngày càng đa dạng

- Mô phỏng song song CPU và MPI.
- Huấn luyện, suy luận AI trên GPU.
- Xử lý dữ liệu quy mô lớn.
- Workflow nhiều giai đoạn.
- Yêu cầu về deadline, chi phí, ưu tiên và chất lượng dịch vụ.

Thực tế giao tiếp hiện nay

Giao diện phổ biến giữa người dùng và scheduler vẫn là một batch script chứa yêu cầu tài nguyên cố định.

Hệ quả

Tài nguyên không đồng nhất, nhưng ngữ nghĩa của nhu cầu người dùng lại bị nén vào một cấu hình duy nhất.

Một resource request đang phải đại diện cho quá nhiều ý nghĩa

Ví dụ batch script

```
#SBATCH -partition=gpu
#SBATCH -gres=gpu:4
#SBATCH -cpus-per-task=16
#SBATCH -mem=64G
#SBATCH -time=02:00:00
```

Scheduler không biết 4 GPU là bắt buộc, bảo thủ, hay chỉ là một lựa chọn để đạt mục tiêu.

Một cấu hình duy nhất thường đồng thời được hiểu là:

- ❶ nhu cầu kỹ thuật tối thiểu;
- ❷ cấu hình mong muốn;
- ❸ giới hạn scheduler được phép chọn;
- ❹ lựa chọn tối ưu của job.

Một request hợp lệ chưa chắc là một request hiệu quả

Intent thực tế

“Tôi muốn job hoàn thành trước 18 giờ. Có thể giảm từ 4 xuống 2 GPU nếu vẫn kịp deadline và tiết kiệm quota.”

Batch script chỉ biểu diễn

$$GPU = 4, \quad walltime = 2h$$

Intent còn chứa

- deadline;
- preference về chi phí;
- khoảng linh hoạt được cho phép;
- điều kiện chấp nhận cấu hình thay thế.

Khi intent không được biểu diễn, decision space bị thu hẹp

Đối với người dùng

- tăng thời gian chờ;
- tiêu thụ quota không cần thiết;
- bỏ lỡ deadline;
- phải thử-sai nhiều lần.

Đối với hệ thống

- giảm khả năng backfilling;
- giữ tài nguyên lớn cho job over-request;
- giảm mức độ đồng thời;
- tăng makespan, giảm utilization và throughput.

Vấn đề cốt lõi

Scheduler không nhất thiết ra quyết định sai; scheduler chỉ được cung cấp một **decision space** quá hẹp.

Từ rigid job đến moldable job

Rigid

Một cấu hình duy nhất:

$$p_j = p_j^{original}$$

Scheduler chỉ quyết định thời điểm và vị trí chạy.

Moldable

Nhiều cấu hình khả thi:

$$\mathcal{P}_j = \{p_{j1}, \dots, p_{jk}\}$$

Chọn profile trước khi job bắt đầu, allocation sau đó giữ nguyên.

Malleable

Tài nguyên thay đổi trong runtime:

$$p_j = p_j(t)$$

Nghiên cứu này tập trung vào **moldability trước submission**, không thay đổi allocation khi job đang chạy.

Moldable scheduling đã chứng minh giá trị của resource flexibility

Các nghiên cứu thường giải bài toán:

$\mathcal{P}_j \rightarrow$ profile selection $\rightarrow$ scheduling

- lựa chọn số processor hoặc node cho job;
- đồng thời chọn profile và thời điểm bắt đầu;
- tối ưu makespan, waiting time hoặc turnaround time;
- multi-resource, QoS-aware và preference-aware scheduling;
- online scheduling cho moldable jobs và workflows.

Kết luận từ literature

Nhiều resource alternatives có thể cải thiện makespan, utilization và throughput khi scheduler được phép nhìn thấy chúng.

Giả định phổ biến: profile set đã được biết

Các phương pháp moldable scheduling thường nhận:

$$\mathcal{P}_j = \{p_{j1}, p_{j2}, \dots, p_{jk}\} \quad \text{hoặc} \quad t_j(p)$$

- profile do application developer khai báo;
- performance curve có được từ profiling;
- đủ historical executions;
- hoặc profile space đã được định nghĩa trong mô hình toán học.

Câu hỏi còn thiếu: Làm sao hình thành các alternatives đáng tin cậy từ một request ban đầu vốn rigid?

Các nhánh related work chưa được nối liền

- ➊ **HPC scheduling:** tối ưu thứ tự, placement và backfilling nhưng thường nhận request đã có cấu trúc.
- ➋ **Moldable scheduling:** khai thác nhiều profile nhưng giả định profile set đã tồn tại.
- ➌ **Prediction và profiling:** dự đoán runtime, memory hoặc utilization nhưng khó với job mới và ít dữ liệu.
- ➍ **Natural-language intent:** chuyển ngôn ngữ tự nhiên thành cấu trúc nhưng chưa chứng minh feasibility.
- ➎ **LLM-assisted scheduling:** có reasoning đa mục tiêu nhưng không tự bảo đảm consistency và policy compliance.

Research gap: tạo ra profile có thể kiểm chứng trước scheduler

Known

Moldable scheduling có thể tối ưu hệ thống khi tập resource alternatives đã được cung cấp.

Limitation

Các phương pháp hiện tại giả định đầu vào đã có cấu trúc, profile set hoặc performance evidence đủ mạnh.

Unknown

Chưa rõ làm thế nào chuyển

rigid request + incomplete intent + cluster constraints + limited evidence

thành tập profiles vừa khả thi kỹ thuật, phù hợp intent, được cho phép và hữu ích cho scheduler.

Gap statement

Phát biểu khoảng trống

Các phương pháp lập lịch cho moldable jobs thường giả định rằng cấu hình tài nguyên khả thi hoặc quan hệ giữa tài nguyên và thời gian thực thi đã biết trước. Giả định này không phù hợp với job mới, ít dữ liệu hoặc người dùng chưa có kinh nghiệm.

- **Gap chính:** methodological gap.
- **Gap về giả định:** assumption gap.
- **Gap hỗ trợ:** empirical gap.

Đích cần đạt

Chuyển request cố định và intent chưa đầy đủ thành tập profile khả thi, nhất quán với intent và có bằng chứng kiểm chứng.

Mục tiêu tổng quát

Xây dựng và đánh giá một phương pháp mô hình hóa ý định người dùng nhằm hình thành các resource profiles khả thi, nhất quán và có thể kiểm chứng trước khi submit job vào hệ thống HPC.

- 1 Xây dựng mô hình biểu diễn intent cho HPC jobs.
- 2 Phân biệt hard constraints, soft preferences và authorized flexibility.
- 3 Hình thành candidate profiles từ intent và cluster capabilities.
- 4 Kiểm tra technical feasibility và intent consistency.
- 5 Xếp hạng profile phù hợp với mục tiêu người dùng.
- 6 Đánh giá tác động trên scheduler hiện hữu.

Câu hỏi nghiên cứu

- ❶ **RQ1 – Intent representation:** Làm thế nào biểu diễn goal, constraint, preference, flexibility và thông tin chưa rõ?
- ❷ **RQ2 – Profile construction:** Làm thế nào chuyển intent, Slurm request, cluster constraints và limited evidence thành các profile khả thi?
- ❸ **RQ3 – Profile selection:** Làm thế nào chọn profile phù hợp khi performance evidence có uncertainty?
- ❹ **RQ4 – Scheduler-level utility:** Profile theo intent có cải thiện hiệu quả scheduler hiện hữu so với request ban đầu hay không?

Trong phạm vi

- batch jobs trên Slurm;
- CPU, GPU, memory, partition và walltime;
- intent đi kèm Slurm script;
- chọn hoặc chỉnh profile trước submission;
- allocation cố định khi job chạy;
- scheduler hiện hữu không bị sửa đổi;
- testbed thực và scheduler simulation.

Ngoài phạm vi

- thay đổi allocation trong runtime;
- phân tích toàn bộ source code;
- tự sửa application để hỗ trợ scalability;
- thiết kế scheduler mới;
- fully autonomous submission không validation.

Intent Intermediate Representation

Với job j , intent được biểu diễn bởi:

$$I_j = (G_j, C_j, F_j, P_j, U_j)$$

- G_j : goal;
- C_j : hard constraints;
- F_j : authorized flexibility;
- P_j : soft preferences;
- U_j : uncertainty và unresolved information.

Ví dụ

“Ưu tiên hoàn thành trước 18 giờ. Có thể dùng từ 2 đến 4 GPU. Nếu cả hai đều kịp deadline thì chọn cấu hình ít tốn quota hơn.”

Intent-conditioned constrained profile search

Mỗi profile có dạng:

$$p = (\text{partition}, \text{nodes}, \text{CPU}, \text{GPU}, \text{memory}, \text{walltime})$$

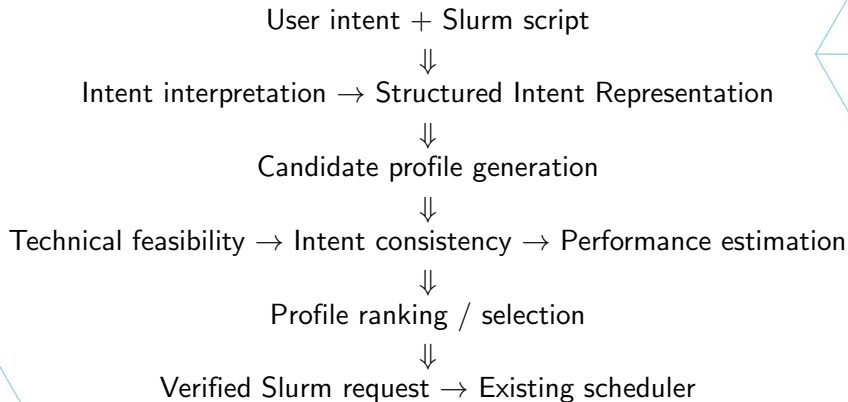
Từ không gian cấu hình cluster $\mathcal{P}$, tìm:

$$\mathcal{P}_j^{ICF} = \{p \in \mathcal{P} \mid C_{site}(p) \wedge C_{workload}(p) \wedge C_{intent}(p, I_j)\}$$

- C_{site} : cluster và site-policy constraints;
- $C_{workload}$: yêu cầu kỹ thuật của workload;
- C_{intent} : intent và authorized flexibility.

Nếu có uncertainty, dùng $\hat{t}_j(p) \pm \epsilon_j(p)$ và chỉ chấp nhận profile an toàn dưới margin này.

Kiến trúc giải pháp: diễn giải bằng LLM, quyết định bằng ràng buộc



Nguyên tắc thiết kế

LLM diễn giải intent; các ràng buộc quyết định được kiểm tra bằng thành phần deterministic.

LLM không phải gap và không phải toàn bộ giải pháp

LLM được sử dụng cho:

natural-language intent $\rightarrow$ structured constraints

Các thành phần phải deterministic:

- parse Slurm directives;
- kiểm tra schema;
- kiểm tra partition và policy;
- sinh candidate profile space.
- kiểm tra feasibility;
- tính service units;
- xác nhận constraint satisfaction;
- log evidence cho audit.

LLM interpretation + constrained search + verification

“Có kiểm chứng” nghĩa là một quyết định có thể audit

Một profile được chấp nhận phải trả lời được:

- ❶ Có hợp lệ trên cluster không? Partition, CPU, GPU, memory và walltime có được hỗ trợ không?
- ❷ Có khả thi đối với workload không? Job có chạy đúng và đạt yêu cầu đầu ra không?
- ❸ Có nhất quán với intent không? Có vi phạm deadline, budget hoặc flexibility không?
- ❹ Vì sao profile này được chọn? Evidence nào hỗ trợ quyết định?

Output

(profile, evidence, constraints, uncertainty)

Không chỉ trả về một resource request, mà trả về một quyết định có thể truy vết.

Từ rigid request đến moldable decision space

Slurm request ban đầu

```
partition=gpu  
gres=gpu:4  
cpus-per-task=16  
mem=64G  
time=02:00:00
```

Intent: cần hoàn thành trước 18 giờ; có thể giảm GPU nếu vẫn kịp và tiết kiệm quota.

Profile	GPU	Runtime	Units	Deadline
p_1	1	5.2 h	thấp	Không đạt
p_2	2	2.7 h	trung bình	Đạt
p_3	4	1.6 h	cao	Đạt

$$\mathcal{P}_j^{ICF} = \{p_2, p_3\} \quad p_j^* = p_2 \text{ nếu ưu tiên tiết kiệm quota}$$

Thiết kế đánh giá tổng thể: ba tầng bằng chứng

- ➊ **Intent correctness:** hệ thống có hiểu đúng goal, constraints, flexibility, preferences, ambiguity và conflict không?
- ➋ **Profile correctness:** hệ thống có tìm đúng $\mathcal{P}_j^{ICF}$ so với ground truth từ profiling không?
- ➌ **Scheduler utility:** profile được chọn có tốt hơn request ban đầu trên cùng scheduler, workload và cluster không?

intent correctness $\longrightarrow$ profile correctness $\longrightarrow$ scheduler utility

E1 — Đánh giá intent interpretation

Mục tiêu: chuyển intent và script thành structured intent đúng và an toàn.

Dataset

- intent đầy đủ, thiếu constraint, mơ hồ;
- xung đột với script;
- nhiều preference;
- không cho phép thay đổi resource.

Baselines

- rule-based parser;
- zero-shot và few-shot LLM;
- LLM + schema + validator.

Metrics

- field-level Precision, Recall, F1;
- exact match và schema validity;
- missing-information detection;
- conflict detection;
- unsupported-field rate;
- premature-commit rate.

E2 — Đánh giá feasible profile search

Ground truth: chạy một tập profile có kiểm soát và ghi nhận:

$$GPU \in \{1, 2, 4\}, \quad CPU \in \{4, 8, 16\}, \quad Memory \in \{16, 32, 64\} GB$$

- chạy thành công hay thất bại;
- output correctness, runtime và peak memory;
- service-unit consumption;
- xác định $\mathcal{P}_j^{GT}$.

Metrics: feasible-profile Precision/Recall, constraint violation rate, top- k recall, Pareto coverage, unsafe recommendation rate và search cost.

E3 — Đánh giá profile selection

Các phương pháp so sánh

- 1 resource request ban đầu;
- 2 profile nhỏ nhất khả thi;
- 3 profile nhanh nhất;
- 4 profile ít service units nhất;
- 5 rule-based selection;
- 6 intent-conditioned selection;
- 7 oracle từ ground truth.

Metrics: execution time, service-unit consumption, deadline/budget satisfaction, intent satisfaction và regret so với oracle.

$$Regret_j = U(p_j^{selected}) - U(p_j^{oracle})$$

E4 — Đánh giá ở cấp scheduler, giữ nguyên scheduler

Hai workload chỉ khác resource profile:

$$W^{original} = \{p_1^{original}, \dots, p_n^{original}\} \quad W^{proposed} = \{p_1^*, \dots, p_n^*\}$$

Giữ nguyên:

- arrival time và job priority;
- scheduling policy và backfilling configuration;
- cluster topology và workload composition.

Metrics: makespan, waiting time, turnaround time, bounded slowdown, utilization, throughput, deadline satisfaction, service units và intent violation rate.

Kịch bản đánh giá scheduler

Tỷ lệ job có flexibility

0%, 20%, 40%, 60%, 80%, 100%

Mức tải

- thấp; trung bình; cao.

Đặc điểm workload

- CPU-dominant; GPU-dominant; mixed;
- nhiều short jobs hoặc long jobs.

Chất lượng evidence

- đầy đủ;
- sparse profiling;
- runtime estimate có noise;
- job mới không có history.

Mục tiêu

Xác định khi nào intent-derived profiles tạo ra giá trị, không chỉ báo cáo một kết quả trung bình.

Ablation study: thành phần nào tạo ra cải thiện?

So sánh hệ thống đầy đủ với các phiên bản:

- không sử dụng intent;
- không sử dụng performance evidence;
- không kiểm tra authorized flexibility;
- không có uncertainty margin;
- không có deterministic verification;
- không có Pareto ranking;
- chỉ sử dụng request ban đầu.

Intent modeling + constrained search + evidence + verification

Testbed thực

Cụm Slurm 5 node NVIDIA V100

- profiling profile space nhỏ;
- kiểm tra correctness;
- controlled experiments;
- validation request rewriting.

LANTA HPC: external validation và portability sang cluster khác.

Simulation

Sử dụng Batsim hoặc simulator tương đương để:

- replay workload lớn;
- thay đổi cluster load và arrival pattern;
- đánh giá makespan, utilization;
- chạy nhiều seed và sensitivity analysis.

Workloads đề xuất

Tiêu chí lựa chọn

- chạy được với ít nhất hai resource profiles;
- có runtime/resource trade-off đo được;
- thời gian chạy đủ ngắn để lặp lại;
- output correctness có thể kiểm tra;
- đại diện cho workload hiện đại trong HPC.

Nhóm workload

- CPU batch workload và MPI mini-application;
- GPU training, inference hoặc batch processing;
- data-processing workload;
- workflow nhiều giai đoạn đơn giản.

Không khẳng định mọi application đều moldable; chỉ xét workload có alternatives hợp lệ.

Đóng góp dự kiến

- ➊ **Intent abstraction:** mô hình phân biệt technical constraints, goals, preferences, authorized flexibility và uncertainty.
- ➋ **Problem formulation:** mô hình hóa intent-to-resource-profile như constrained search dưới limited evidence và uncertainty.
- ➌ **Phương pháp:** hình thành intent-consistent feasible profiles từ request rigid ban đầu.
- ➍ **Benchmark:** liên kết Slurm request, intent, cluster constraints, evidence, feasible profiles và scheduler workload.
- ➎ **Empirical evidence:** đánh giá tác động lên scheduler hiện hữu mà không thay đổi scheduling algorithm.

Khác biệt so với các hướng hiện có

Hướng	Đầu vào	Đầu ra	Hạn chế liên quan
Traditional scheduling	Rigid request	Thời điểm và placement	Không thấy alternatives
Moldable scheduling	Profile set đã biết	Profile và schedule	Giả định alternatives tồn tại
Resource prediction	Historical logs	Runtime/resource estimate	Khó với job mới
LLM-only assistant	Natural language	Resource suggestion	Không bảo đảm feasibility
Nghiên cứu đề xuất	Intent + request + constraints + evidence	Verified feasible profile set	Xử lý bước còn thiếu trước scheduler

Điểm khác biệt không phải là dùng LLM, mà là tạo ra một **decision space có cấu trúc và có thể kiểm chứng**.

Intent không đủ thông tin

- Không tự suy diễn hard constraint chưa được cho phép.
- Đánh dấu unresolved fields hoặc yêu cầu xác nhận.

Performance estimate sai

- Dùng uncertainty margin và conservative feasibility checking.

Profile chạy được nhưng output sai

- Dùng workload-specific correctness test.

Giá trị phụ thuộc workload

- Đánh giá theo tỷ lệ flexible jobs và workload class.

Lộ trình thực hiện

- ❶ **Mô hình và dữ liệu:** hoàn thiện intent schema, profile representation, workloads, profile space và ground truth.
- ❷ **Phương pháp:** intent interpretation, candidate generation, feasibility checking, estimation, ranking và evidence logging.
- ❸ **Đánh giá thành phần:** intent benchmark, profile-search evaluation, sparse-evidence experiment và ablation.
- ❹ **Scheduler evaluation:** simulation, workload replay, real-cluster validation và robustness analysis.

model/data → method → component evaluation → scheduler evaluation

Kết luận: làm cho intent trở thành đầu vào có giá trị

Scheduler hiện tại không nhất thiết thiếu khả năng tối ưu; nó thường thiếu thông tin về **những lựa chọn tài nguyên mà người dùng thực sự chấp nhận**.

Rigid request + User intent + Cluster constraints + Limited evidence



Intent-consistent feasible profiles



Verified submission to an unchanged scheduler

Thông điệp chính

Bước còn thiếu giữa “hiểu người dùng” và “lập lịch” là tạo ra decision space vừa phù hợp intent, vừa khả thi và có thể kiểm chứng.

Tóm tắt đóng góp

1. Mô hình hóa intent

Xác định goal, constraint, preference và flexibility.

2. Tìm kiếm có ràng buộc

Tạo các lựa chọn tài nguyên hợp lệ từ request rigid.

3. Kiểm chứng quyết định

Bảo đảm profile phù hợp workload, cluster và intent.

4. Đánh giá cấp scheduler

Chứng minh giá trị ở hiệu quả vận hành, không chỉ ở khả năng hiểu ngôn ngữ.

Từ “hiểu người dùng” đến “cải thiện hệ thống”.